Secure Microservices Inventory System

Final Project Report
MCSC 6002, Cybersecurity Architecture, Design, and Secure Software Development
Arman Akhmetov (N10045688), Mario Silveira

1. Introduction and Objectives

Splitting an application into microservices gives an attacker more targets: every service, the gateway in front of them, the message broker between them and each database. A common weakness is trusting the internal network, so that once a request is past the edge, nothing else checks it. For an inventory system this matters directly, because whoever can change stock levels or product records can cause financial loss.

This project follows Option 2 (Secure System Architecture and Prototype). We built an inventory system from three .NET services behind an Ocelot API gateway, with RabbitMQ events between the services. The objectives, taken from the proposal, were:

- authenticate every request at the gateway and check permissions again in each service;
- use short-lived signed access tokens with rotating refresh tokens;
- protect traffic between the gateway and the services, and the integrity of queue messages;
- validate all input;
- model the threats with STRIDE and test the controls against them.

2. System Overview

The Identity service manages users, roles and permissions and issues tokens. The Product service manages brands and products; when a product is created or deleted it publishes an event. The Inventory service consumes these events and creates or removes the matching stock items, so the two services stay consistent without calling each other (eventual consistency). All client traffic enters through the API gateway.

Each service has its own SQL Server database. The Product service caches its product list in Redis. The services use .NET 10 with Clean Architecture, CQRS through MediatR, Entity Framework Core, FluentValidation and ASP.NET Core Identity, and MassTransit for RabbitMQ. Two shared NuGet libraries hold the permission constants and the request and response contracts. For local runs, SQL Server, RabbitMQ and Redis start with Docker Compose and the services run with `dotnet run`.

Changes from the proposal

Three points differ from the proposal. Docker Compose runs SQL Server, RabbitMQ and Redis, but the four services run on the host, started together by one script (`scripts/run.sh`); container images for the services are future work. Users cannot register themselves: an administrator creates accounts, which suits an internal company inventory system. Queue messages are not encrypted field by field. The connection to RabbitMQ uses TLS, and events carry only product ids, names and quantities, so TLS in transit and the HMAC signature cover the risk.

3. Architecture Design

Figure 1 shows the network zones and which links are encrypted. Figure 2 is a level 1 data flow diagram with the three main flows: login, product creation (including the event to Inventory) and stock queries. Figure 3 marks the five trust boundaries used in the threat model.

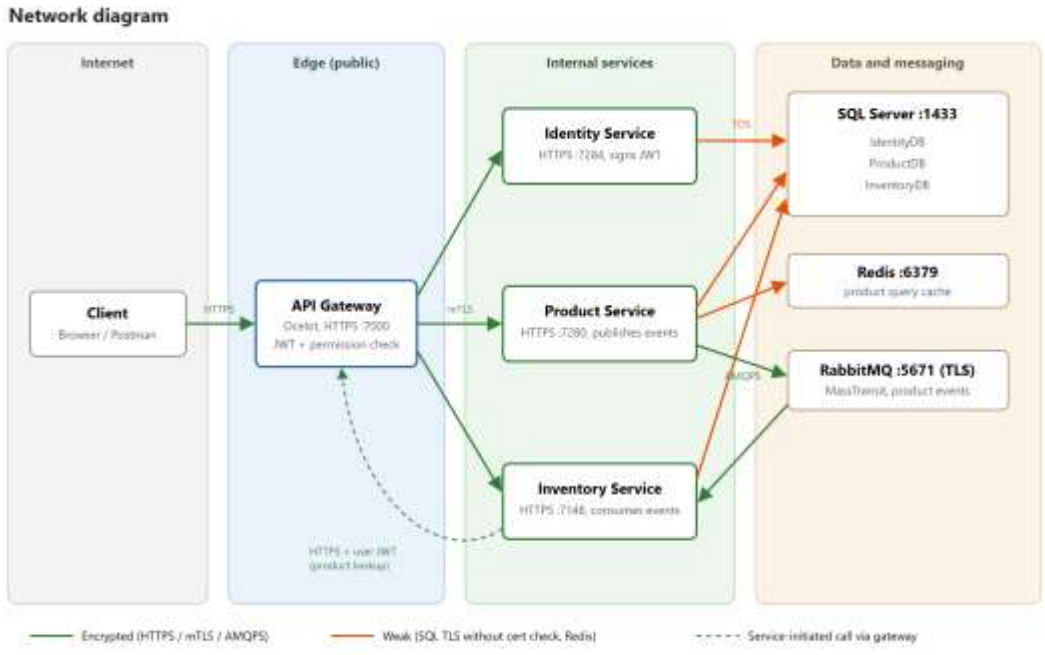


Figure 1. Network diagram

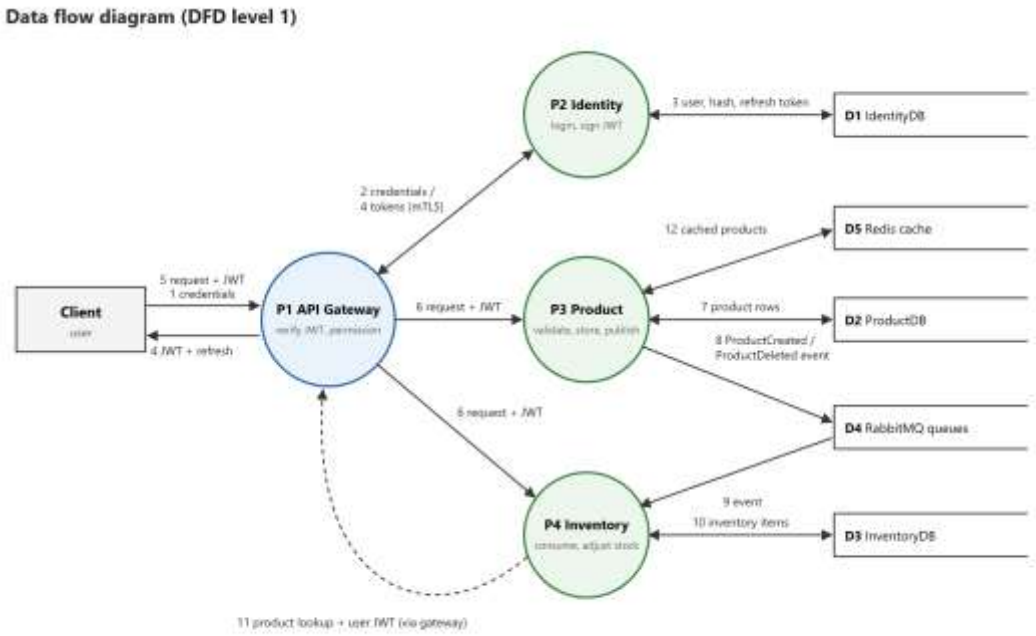


Figure 2. Data flow diagram

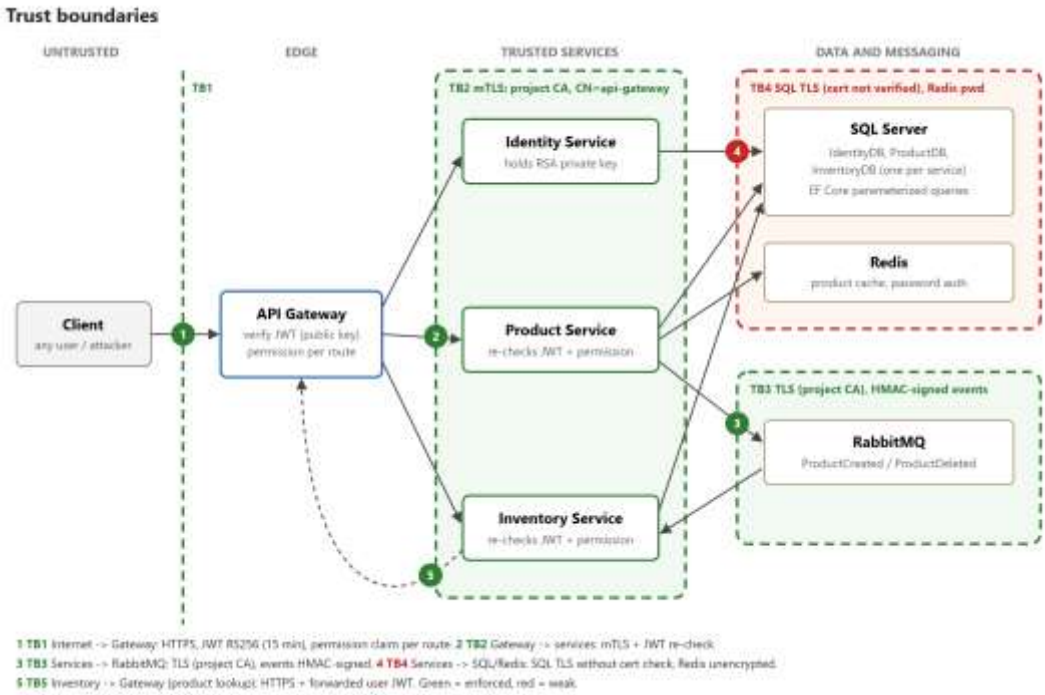


Figure 3. Trust boundaries

The design follows three principles. Defense in depth: the gateway checks every token and permission, and each service checks them again, so a misconfigured route does not open

a service. Least privilege: only the Identity service holds the token signing key. No implicit trust of the internal network, in line with zero trust architecture (Rose et al., 2020): services accept connections only from the gateway, and queue events must be signed.

4. Security Features Implemented

4.1 Authentication and access control

Access tokens are JWTs signed with RS256 and valid for 15 minutes. The Identity service signs them with an RSA private key; the gateway and the other services receive only the public key, so a compromised service cannot issue tokens. Every validator accepts only RS256, which blocks algorithm confusion attacks such as `alg=none` or HS256 (Sheffer et al., 2020). Refresh tokens are 32 random bytes, valid for one day and replaced on every use, so a used refresh token is rejected. On logout a user can revoke their refresh token.

Authorization is permission based. The gateway checks the permission claim required by each route before forwarding the request, and the service checks the same permission again with its own `[HasPermission]` attribute. Passwords must have at least 12 characters with 6 different ones, following the length-over-composition advice in NIST SP 800-63B (Grassi et al., 2017). After 5 failed logins an account is locked for 15 minutes, and the gateway allows at most 5 login or refresh requests per minute per IP address. Wrong passwords, unknown users and locked accounts all get the same message, so a response does not reveal whether an account exists or is locked.

4.2 Secure communication

Clients reach the gateway over HTTPS, with HSTS outside development. Between the gateway and the services we use mutual TLS. A project certificate authority issues a client certificate to the gateway, and each service accepts only a client certificate issued by that CA with the name `api-gateway`. The gateway in turn accepts only service certificates from the same CA and checks the host name. The services no longer listen on plain HTTP, so even a host inside the network cannot call a service directly. The connection to RabbitMQ also uses TLS: the broker listens only on its TLS port, with a certificate from the same CA, and the services check that certificate and its host name.

4.3 Hashing and integrity

Passwords are stored as salted PBKDF2 hashes by ASP.NET Core Identity. Refresh tokens are stored only as SHA-256 hashes, so a copy of the database holds no usable session. Queue events carry an HMAC-SHA256 signature (Krawczyk et al., 1997) over the event type, a timestamp and the message body. The Inventory service verifies the signature in a filter that runs before every consumer and rejects events that are unsigned, altered or older

than five minutes. Rejected events go to an error queue as evidence instead of being retried.

4.4 Input validation

Request bodies in all three services are validated with FluentValidation for type, length and range, and the Inventory service accepts only known stock statuses. The gateway rejects bodies larger than 2 MB. Entity Framework Core sends all queries with parameters, which prevents SQL injection. Product quantity is limited to 1 to 1000, because the Inventory service creates one record per unit. The consumer checks the same limit again, in case an event bypasses the Product service.

4.5 Secrets and error handling

No password or key is committed. Infrastructure and seed passwords live in a git-ignored `.env` file and in each service's .NET user secrets; signing keys and certificates are kept outside all repositories. RabbitMQ uses its own account instead of the default guest user, Redis requires a password and SQL connections are encrypted, although the server certificate is not yet validated. The gateway returns generic error messages, also in place of server errors from the services, logs the details server side and removes headers that reveal the server software. It also adds `X-Content-Type-Options`, `Cross-Origin-Resource-Policy` and `Cache-Control: no-store`.

5. Threat Modelling and Mitigation

We applied STRIDE (Shostack, 2014) to each component and trust boundary and rated each threat by likelihood and impact (Low, Medium or High). The model lists 26 threats; Appendix A has the full table. At the end of the project, 17 are mitigated, 5 partly mitigated, 3 planned and 1 not applicable.

The highest risks at the start were forged tokens (a signing secret shared by four services and committed to git), direct calls to services that bypass the gateway, password guessing, forged or altered queue events and an unbounded quantity that let one event create any number of records. All of these are now mitigated and tested.

Broken object level authorization, the first item in the OWASP API Security Top 10 (OWASP Foundation, 2023), was a focus of the proposal. In the final data model no record belongs to a single user, so the threat does not apply; the related function level checks are covered by threat E1 and tested.

The main remaining risks are:

- a leaked refresh token can be used once, because reuse is not detected;

- a signed event can be replayed within the five-minute window;
- the HMAC key is shared, so the Inventory service could also forge events;
- there is no audit log of who changed products or stock;
- account lockout lets an attacker lock a known account on purpose.

6. Testing and Validation

We tested in four ways. A scripted suite of 37 attack cases (Appendix B) runs against the live system, and each case maps to a threat ID. Semgrep scanned the source of all seven repositories, Gitleaks scanned their git history and OWASP ZAP ran an authenticated active scan of all 19 gateway routes.

All 37 attack cases pass. They cover forged, expired and wrongly signed tokens, privilege escalation, refresh token reuse and use after logout, rate limiting and lockout, direct calls to services without the gateway certificate, plain connections to the message broker, SQL injection and oversized or malformed input, and events published straight to RabbitMQ by an attacker with broker credentials.

Semgrep reported five findings: four GitHub Actions referenced by a tag instead of a commit, which we accept as a low supply chain risk, and the deliberate `ValidateLifetime = false` in the refresh flow. Gitleaks found the old HS256 secret in the history of four repositories; it is no longer used and test A4 shows it is rejected. ZAP found no high risk issues. Its one medium alert is a false positive caused by our own SQL injection test data, and the missing HSTS header is expected in development mode.

Testing also found real defects, which we fixed and retested. Forged tokens crashed the request instead of returning 401, because the framework placed the authentication middleware before our header middleware. The product list stayed cached for an hour after changes. ZAP reported three missing response headers and an unneeded static page on the gateway. A final end-to-end check found that the Inventory service could not call the Product service through the gateway, and that the resulting error page leaked a stack trace and the caller's token; the gateway now uses a certificate from our own authority and hides error bodies from the services. A restore on a clean machine also showed that our shared libraries were not on nuget.org, where unrelated packages use the same names: the build silently took a stranger's AuthLibrary 2.0.0. Each service now ships the exact packages and maps those names to them only.

7. Challenges and Lessons Learned

The largest delay came from dependencies. MassTransit 9 requires a commercial license, and two services failed at startup until we moved to version 8, which is open source.

MediatR 14 has a similar license that allows only development use. Checking licenses early would have saved time.

Several defects appeared only when we attacked the running system: the middleware ordering crash and the stale cache both passed the build and normal use. Running negative tests early is worth the effort. We also learned to verify the tools themselves. Our first ZAP scan never sent the token and tested only the login wall, which we noticed only by checking the gateway logs.

8. Conclusion

The prototype delivers the security controls promised in the proposal: RS256 tokens with short lifetimes and hashed, rotating refresh tokens, permission checks at the gateway and in every service, mutual TLS between them, TLS to the message broker, signed queue events, input validation and protection against password guessing. The threat model and the test results show how each control maps to a threat and that it works. The remaining work is refresh token reuse detection, protection against replayed events, an audit log and container images for the services themselves.

Team Contributions

Mario Silveira built the Identity, Product and Inventory services, the shared libraries and their build pipelines. Arman Akhmetov built the token and gateway security (RS256 signing and validation, route permissions, mutual TLS, rate limiting, lockout and password policy, refresh token hashing and revocation, error handling), the message signing and broker TLS, the local infrastructure, the threat model and diagrams, and the testing, and wrote this report.

References

Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). Digital identity guidelines: Authentication and lifecycle management (NIST Special Publication 800-63B). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63b

Krawczyk, H., Bellare, M., & Canetti, R. (1997). HMAC: Keyed-hashing for message authentication (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

OWASP Foundation. (2023). OWASP API security top 10 2023. https://owasp.org/API-Security/editions/2023/en/0x00-header/

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero trust architecture (NIST Special Publication 800-207). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207

Sheffer, Y., Hardt, D., & Jones, M. (2020). JSON Web Token best current practices (RFC 8725). Internet Engineering Task Force. https://doi.org/10.17487/RFC8725

Shostack, A. (2014). Threat modeling: Designing for security. Wiley.

Appendix A. STRIDE Threat Model

ID	STRIDE	Component	Threat	Risk	Mitigation	Status
S1	Spoofing	All	Forged JWT using shared HS256 secret (was in 4 configs)	High	RS256; private key only in Identity, others get public key	Done
S2	Spoofing	All	Algorithm confusion (none, HS256, RS384)	High	ValidAlgorithms pinned to RS256 (incl. refresh)	Done
S3	Spoofing	Services	Direct call bypassing gateway	High	mTLS; services reject any client cert except gateway's	Done
S4	Spoofing	Identity	Password brute force / weak passwords	High	Gateway 5 req/min per IP on /auth/*; lockout after 5 failures (15 min); min 12 chars, 6 unique (NIST 800-63B), enforced on user creation	Done
S5	Spoofing	RabbitMQ	Fake events published by anyone with broker access	High	Dedicated broker user from user-secrets; Inventory accepts only events HMAC-signed with a key shared by Product and Inventory	Done
T1	Tampering	Gateway, services	Edited JWT payload (add permission)	High	Signature validated at gateway and again in service	Done
T2	Tampering	TB2	MITM between gateway and services	High	TLS + mTLS, custom CA trust, hostname check	Done
T3	Tampering	RabbitMQ	Modified or replayed event (Quantity, ProductId)	High	HMAC over type + timestamp + body, checked in a consume filter; rejects go to the _error queue (done). AMQP over TLS with project CA cert, plain port closed (done). Replays within 5 min (planned)	Partial
T4	Tampering	Services, DB	SQL injection	Medium	EF Core parameterized queries; FluentValidation in all three services (Inventory: known statuses only)	Done
T5	Tampering	Build	Dependency confusion: shared AuthLibrary/ResponseLibrary are not on nuget.org, which hosts unrelated packages with the same names; a clean restore pulled AuthLibrary 2.0.0	High	Exact packages bundled in each repo (nuget-local); nuget.config packageSourceMapping limits both names to that folder	Done
R1	Repudiation	Product, Inventory	No record of who changed products or stock	Medium	Audit log with user id (NameIdentifier) per write	Planned
I1	Info disclosure	Gateway	Exception messages and internal URLs in responses	Medium	Generic messages; details to server log	Done
I2	Info disclosure	Services	Dev exception page (stack trace) passed through gateway	Medium	Gateway replaces every downstream 5xx body with a generic message; direct calls blocked by mTLS	Done
I3	Info disclosure	Gateway	Server header fingerprinting	Low	Server/X-Powered-By stripped, incl. downstream copies	Done
I4	Info disclosure	Repos	Secrets in git: seed passwords, RabbitMQ creds, old HS256 secret in history	High	DB/broker/Redis/seed passwords moved to user-secrets (.env, git-ignored); HS256 secret in	Partial

ID	STRIDE	Component	Threat	Risk	Mitigation	Status
					history but unused; all services share SQL sa login	
I5	Info disclosure	Identity DB	Refresh tokens stored in plaintext	Medium	Only the SHA-256 hash is stored; constant-time compare	Done
I6	Info disclosure	JWT	Email and phone in readable token payload	Low	Keep only id and permissions	Planned
I7	Info disclosure	TB4	SQL traffic unencrypted	Medium	Encrypt=True (done); trusted server cert instead of TrustServerCertificate=True (planned)	Partial
D1	DoS	Inventory	ProductCreated.Quantity unbounded; consumer creates one row per unit	High	Quantity 1..1000 in Product validators and again in the Inventory consumer	Done
D2	DoS	Gateway	No request rate limiting	Medium	Per-IP limit on /auth/* (done); other routes (planned)	Partial
D3	DoS	All	Oversized bodies (Kestrel default ~28 MB)	Low	Gateway MaxRequestBodySize 2 MB; services reachable only through the gateway	Done
E1	EoP	Gateway, services	Basic user calls admin endpoint (BFLA)	High	Permission claim at gateway (RouteClaimsRequirement) + [HasPermission] in service	Done
E2	EoP	Clients	Stolen access token reused	Medium	15-min lifetime; refresh token rotated on each use	Done
E3	EoP	Identity	Old access token + leaked refresh token -> new session	Medium	Rotation, 1-day expiry, revoke on logout (/auth/revoke-token) (done); reuse detection (planned)	Partial
E4	EoP	Services	BOLA/IDOR on records	Low	No per-user records in current model; revisit if ownership added	N/A
E5	EoP	mTLS	Stolen gateway cert (PFX without password, no revocation)	Medium	Restrict file access, Docker secrets, short validity; CRL if needed	Planned

Appendix B. Security Test Results

ID	Threat	Case	Expected	Result
A1	S1	Request without token	401	401
A2	T1	Token payload edited to add Create permission	401	401 Invalid token
A3	S2	Token with alg=none	401	401 Invalid token
A4	S1	HS256 token signed with the old (leaked) shared secret	401	401 Invalid token
A5	S2	RS384 token signed with the real private key	401	401 Invalid token
A6	E2	Expired RS256 token	401	401 Token expired
A7	E1	Basic user creates a product	403	403 at gateway
A8	E1	Basic user reads products	200	200
A9	E3	Refresh token used, then reused	200, then rejected	200, then 400
A10	S4	Wrong password vs unknown user	Same message	Both "Invalid credentials."
A11	S4	6th login attempt within one minute	429	429

ID	Threat	Case	Expected	Result
A12	S4	Create user with "short1" and "aaaaaaaaaaaa"	400, 400	400 (length), 400 (unique chars)
A13	S4	5 wrong passwords, then the correct one	Correct one rejected	Rejected (locked 15 min)
A14	E3	Logout (revoke), then refresh	200, then rejected	200, then 400
N1	S3	Direct call to a service, no client certificate	TLS handshake fails	Handshake fails
N2	S3	Direct call with self-signed cert CN=api-gateway	TLS handshake fails	Handshake fails
N3	T2	Plain HTTP to a service	No listener	Connection refused
N4	T3	RabbitMQ: plain AMQP port, then TLS port	5672 closed; 5671 TLS, project CA cert	Closed; TLS 1.3, project CA
V1	T4	SQL injection in login (`' OR '1'='1' --`)	Rejected, no 500	400 Invalid credentials
V2	T4	SQL injection as product name	Stored as text	Stored literally, table intact
V2b	-	New product appears in the (cached) product list	Listed	Listed
V3	T4	SQL injection in route id (`1 OR 1=1`)	400/404	400
V4	D3	Description of 10,000 characters	400	400 (max 400)
V5	D1	Quantity 5000 and -1	400, 400	400, 400
V6	T4	Quantity sent as a string	400	400
V7	T4	`<script>` payload as product name	Stored as data, served as JSON	application/json
V8	T4	Unknown inventory status (`<script>`)	Rejected	404, allowed values listed
V9	D3	1 MB, then 3 MB request body (2 MB limit)	400 (validator), then 413	400, then 413
M1	S5	Unsigned event published directly to RabbitMQ	Rejected	0 items, error queue
M2	T3	Event signed for qty 1, sent with qty 500	Rejected	0 items, error queue
M3	T3	Validly signed event, 10 minutes old (replay)	Rejected	0 items, error queue
M4	D1	Validly signed event, qty 5000	Rejected	0 items, error queue
M5	-	Validly signed event, qty 2 (control)	Accepted	2 items
I1	I3	Server header on proxied response	Absent	Absent
I2	I1	Malformed JSON body	400, no stack trace	400, no stack trace
I3	I1	API security headers	nosniff, same-origin, no-store	All three present
I4	I3	Static page at `/`	404	404